USEI07 – Build a Balanced 2D KD-Tree

Context

A 2D KD-tree enables efficient execution of:

- **Range queries**
- **Nearest-neighbour searches**
- **k-nearest neighbour queries**

These operations become significantly faster than scanning all stations sequentially.

User Story

As a data engineer, I want to build a balanced 2D KD-tree from coordinate pairs (lat, lon) so that geographic searches are efficient and predictable.

Acceptance Criteria

- The tree must be constructed using a **bulk-build strategy** to ensure balance.
- The build process must reuse the **AVL/BST latitude and longitude trees** from USEI06.
- If multiple stations share the same coordinates, e.g.
  (38.71387, -9.122271) belonging to *Lisbon Santa Apolónia* and *Lisbon Oriente*,
  they must be stored together in a **bucket**, sorted alphabetically by station name.
- The output must include:
  - Total number of nodes
  - Tree height
  - All distinct bucket sizes
  - A temporal complexity analysis

Temporal Complexity Analysis

1. KD-Tree Construction

The tree is built recursively:

1. Sort stations according to the current axis (latitude or longitude).
2. Select the median as the splitting point.

3. Partition the remaining stations into left and right sublists.
4. Recurse while alternating the splitting axis.

Detailed Complexity Analysis (Code Excerpt)

```
private Node buildBalancedRecursive(List<Station> stations, int depth) {
    if (stations.isEmpty()) return null;

    if (splitByLatitude){
        // ...
        stations.sort(comparator); // O(n log n) per call
        // ...
    }
    int medianIndex = stations.size() / 2;
    // Distribute stations into left, right, bucket
    for (int i = 0; i < stations.size(); i++) // (...) O(n) per call

        // Recursively build subtrees
        node.left = buildBalancedRecursive(leftStations, depth + 1);
    node.right = buildBalancedRecursive(rightStations, depth + 1);

    return node;
}
```

Current Complexity

Because sorting happens at every recursive call:

$O(n \log^2 n)$

Optimal Complexity (if pre-sorted arrays were maintained)

$O(n \log n)$

Summary Table

Operation	Expected Complexity	Worst Case
KD-tree Build	$O(n \log n)$	$O(n \log^2 n)$